



武汉大学

WUHAN UNIVERSITY

数据结构与程序设计

(Python语言)

DATA STRUCTURE AND PROGRAM DESIGN IN PYTHON

第6讲 函数（一）



Python课程组



2025年02月



CONTENTS

目 录

- | | | | |
|---|---------|---|-----------|
| 1 | 定义和调用函数 | 5 | 函数定义的补充内容 |
| 2 | 程序模块化 | 6 | 传递列表 |
| 3 | 传递参数 | 7 | 实践指导 |
| 4 | 函数返回值 | | |



01

定义和调用函数



引例：图形绘制程序

■ 编写一个可以根据用户的需求绘制各种图形的程序

✿ 图形的形状包括矩形、直角三角形、平行四边形和菱形等，图形的大小（高度、宽度）由用户指定

➤ 字符模式下的各种图形

*

*

**

*

图形1

图形2

图形3

图形4

图形5

➤ 观察这些图形的构成，它们都由若干行星号组成。例如，图形1是高3宽8的矩形，即由3行8列共24个星号构成



引例：图形绘制程序



■ 绘制3个大小不同的矩形的代码实现

```
# 绘制3行8列的星号矩形
for row in range(3):
    print("*"*8)

print()

# 绘制3行4列的星号矩形
for row in range(3):
    print("*"*4)

print()

# 绘制5行3列的星号矩形
for row in range(5):
    print("*"*3)
```

程序执行结果：

```
*****
*****
*****

****
****
****

***
***
***
***
***
```



定义和调用函数



■ 为了避免重复代码，可以通过定义和调用函数，改进引例程序

- ✱ 定义一个名称为draw_rect的函数，功能是绘制指定大小（高度和宽度）的星号矩形，高度和宽度是绘制矩形的参数。
- ✱ 矩形大小（高度和宽度）在每次调用该函数时指定。

```
# 定义函数draw_rect
# 绘制指定大小（高height宽width）的矩形
def draw_rect(height, width):
    for row in range(height):
        print("*"*width)
```

先定义函数

再调用函数
一共调用了3次，每次
指定不同的大小

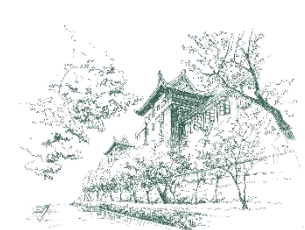
```
# 调用函数draw_rect
# 绘制3行8列的星号矩形
draw_rect(3, 8)

print()

# 绘制3行4列的星号矩形
draw_rect(3, 4)

print()

# 绘制5行3列的星号矩形
draw_rect(5, 3)
```

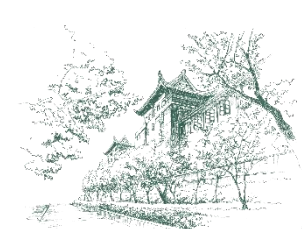



函数的概念

■ 函数是完成某一种特定任务的一段程序代码，只需定义一次，就可以多次调用。

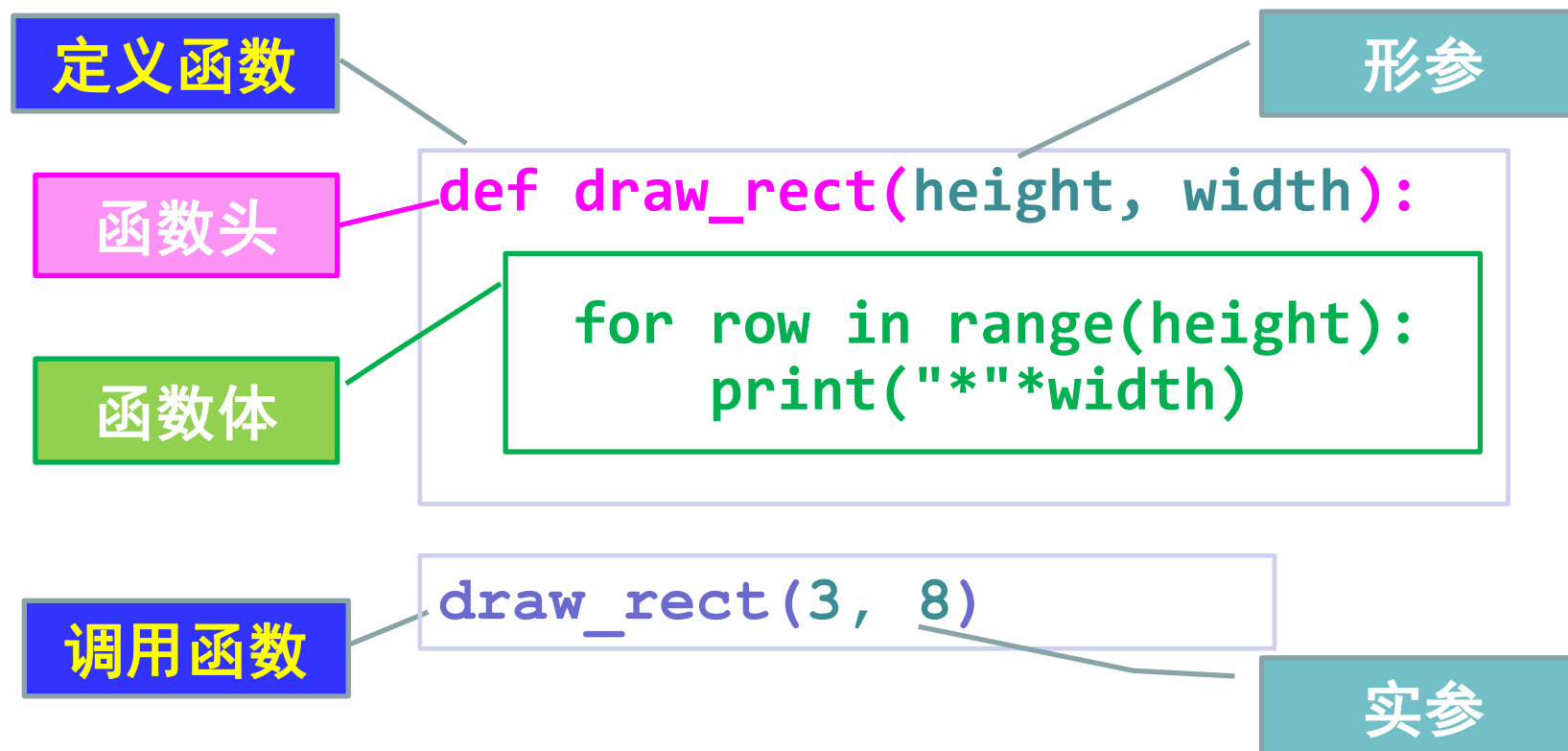
- ✱ 通过使用函数，可以将复杂任务分解为更小、更易于理解和维护的程序模块（函数），程序的结构更清晰。
- ✱ 函数提高了代码的复用性，当函数被定义后，可以在其他地方轻松地调用。
- ✱ 注意：函数定义并不会执行函数体，函数只有当被调用时才会执行。

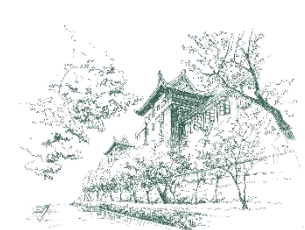
提醒：同学们在完成函数这部分的实践作业时，除了定义相应的功能函数模块，还应调用函数，执行得到运行结果



定义和调用函数

■ 引例：定义绘制星号矩形的函数，并调用该函数





函数定义



■ 用def语句定义函数，包括函数头和函数体：

def 函数名([形式参数表]):
函数体

✿ 说明

- 函数头由关键字**def**加空格打头，后接一个标识符作为函数名，接着是一对圆括号（里面是用逗号隔开的参数表），最后有一个冒号。
- 冒号后面换行，相对于**def**向右缩进的若干行**Python**语句构成函数体。
- **形参**是形式参数的简称，函数调用时，形参用来接收传递过来的实参数据。
- 形参之间用逗号分隔，形参不需要声明类型，也不需要指定函数返回值类型。
- 不管有无形参，函数名后的圆括号和冒号必不可少。



函数调用



■ 只有当函数被调用时，才会执行函数的代码。函数调用的语法格式：

函数名([实际参数表])

※ 说明

- 函数定义时若有参数，必须在调用时提供实际参数，简称**实参**。
- 实参的个数、位置要与函数定义时的形参相对应。
- 函数调用的形式：
 - 直接以语句形式出现
 - 在表达式中出现
 - 作为另一个函数调用的实参

```
print(values)

area = 3.14 * pow(radius, 2)

print( list( range(1,10) ) )
```

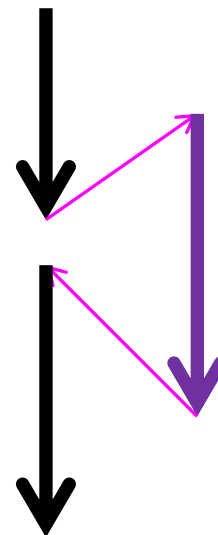


函数调用的过程



■ 当函数调用发生时:

- ✿ 调用程序暂停
- ✿ 函数形参被赋值为实参(按位置对应)
- ✿ 执行函数体
- ✿ 控制返回调用点的下一条语句





函数的定义与调用举例

```
def main():  
    sing("Fred")  
    print()  
    sing("Lucy")  
  
def sing(person):  
    happy()  
    happy()  
    print("Happy birthday, dear", person + ".")  
    happy()  
  
def happy():  
    print("Happy Birthday to you!")
```

Diagram illustrating function calls: An arrow points from `sing("Fred")` in `main` to the `def sing(person):` block. Another arrow points from the `happy()` call inside `sing` to the `def happy():` block.

person: "Fred"

```
def main():  
    sing("Fred")  
    print()  
    sing("Lucy")  
  
def sing(person):  
    happy()  
    happy()  
    print("Happy birthday, dear", person + ".")  
    happy()
```

Diagram illustrating function calls: An arrow points from `sing("Fred")` in `main` to the `def sing(person):` block. Another arrow points from the `happy()` call inside `sing` to the `happy()` call inside `main`.

```
def main():  
    sing("Fred")  
    print()  
    sing("Lucy")  
  
def sing(person):  
    happy()  
    happy()  
    print("Happy birthday, dear", person + ".")  
    happy()
```

Diagram illustrating function calls: An arrow points from `sing("Lucy")` in `main` to the `def sing(person):` block. Another arrow points from the `happy()` call inside `sing` to the `happy()` call inside `main`.

person: "Lucy"



武汉大学
WUHAN UNIVERSITY

02

程序模块化



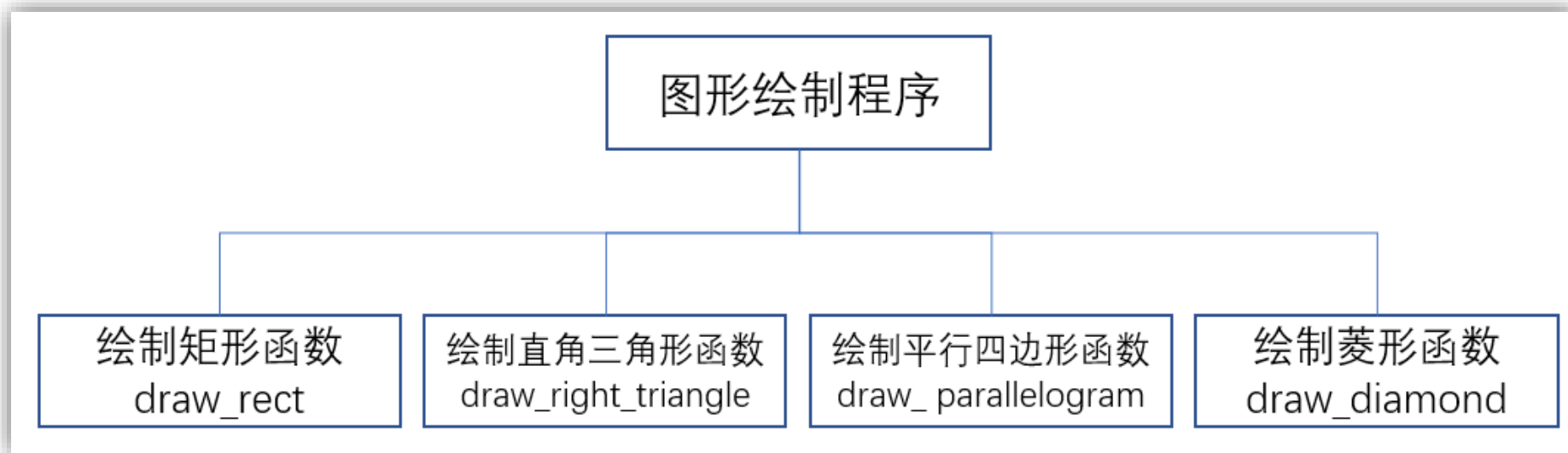
程序模块化



武汉大学
WUHAN UNIVERSITY

■ 能够绘制用户指定大小的各种图形的程序，可以设计为下图所示的结构

- ✿ 程序包含多个函数，每个函数专门绘制一种形状的图形。
- ✿ 需要绘制一个具体的图形时，调用相应的函数。
- ✿ 函数是构成程序的最简单的模块，包含多个函数的程序也被称为多模块程序。





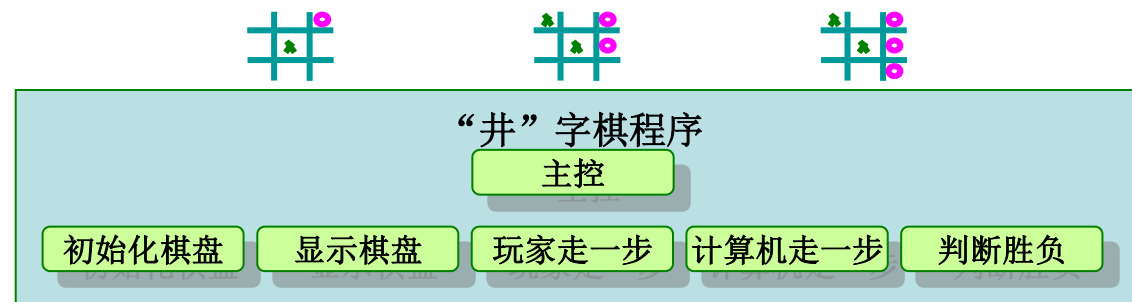
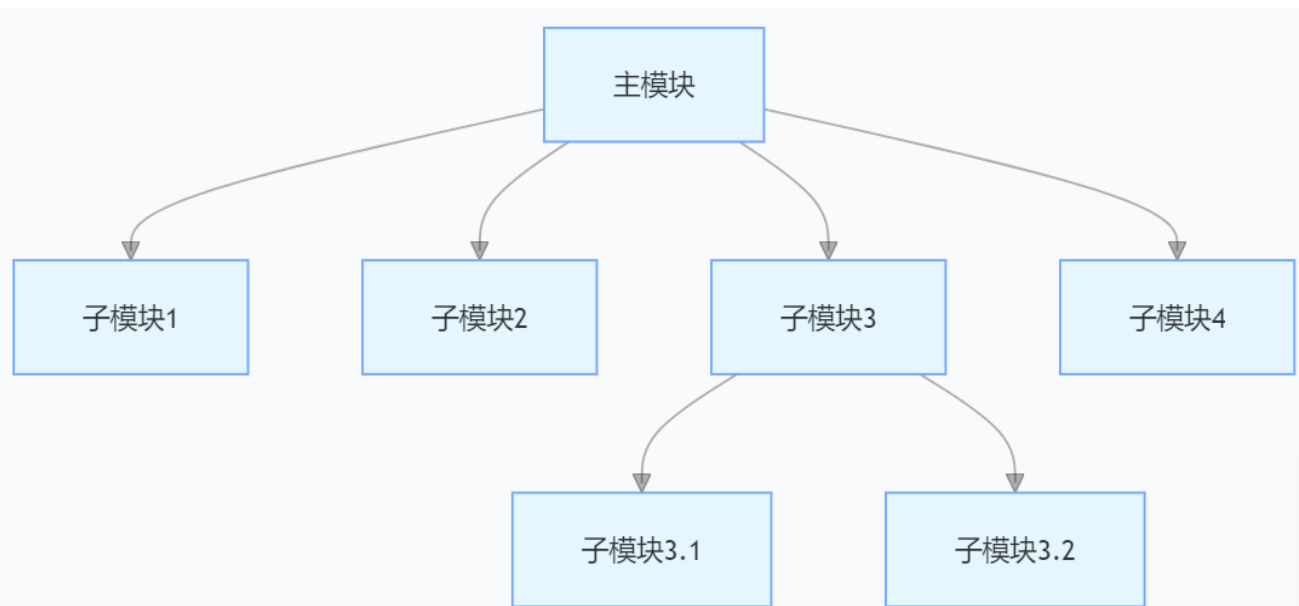
程序模块化



武汉大学
WUHAN UNIVERSITY

分而治之，自顶向下，逐步细化

- 在设计程序时，依据需求对程序要完成的任务（或者称为要求解的问题）进行分解，把完成每一个任务的程序代码定义为一个函数。
- 对于复杂程序，一个模块要完成的任务往往需要进一步分解为若干子任务，如果子任务还不够简单，则可以继续进行分解，直到各个子任务足够具体。
- 这种设计程序的过程被称为程序模块化。





程序模块化：分而治之，自顶向下



武汉大学
WUHAN UNIVERSITY

■ 程序模块

- ✱ 程序模块可能包含多个自定义函数、自定义类、全局变量等。
- ✱ 在Python中，把一个.py程序文件称为一个模块，在程序文件中可以定义函数、全局变量、类、对象等。



通过函数实现程序模块化

■ 函数是实现结构化程序必不可少的工具

- ✿ 将可能需要反复执行的代码**封装**为函数，并在需要该功能的地方进行**调用**。
- ✿ 不仅可以**实现代码复用**，更重要的是可以**保证代码的一致性**，只需要修改该函数代码则所有调用均受到影响。

■ 设计函数时，应注意提高模块的内聚性，同时降低模块之间的隐式耦合。

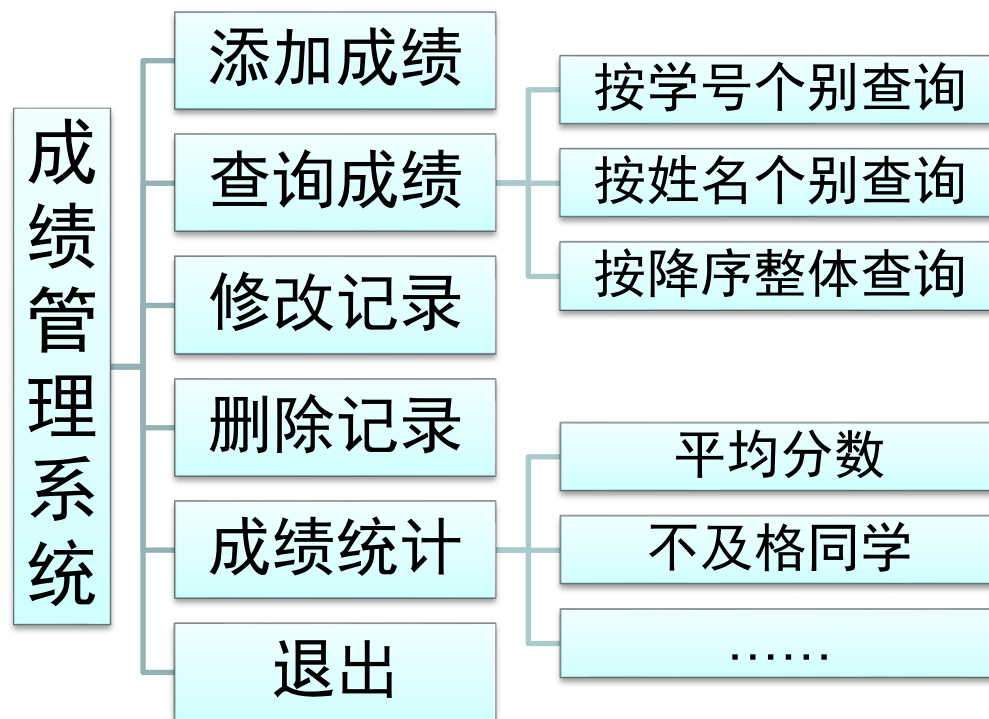
试一试在图形绘制程序中定义另外3个绘图函数（绘制直角三角形、平行四边形和菱形的函数），然后增加交互功能，根据用户的输入绘制不同形状的图形。

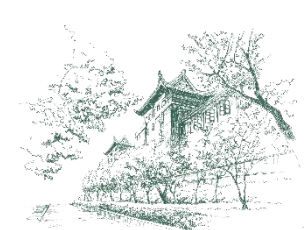


模块化后的程序结构示例



■ 学生成绩管理系统的程序模块化结构





Python函数的类型

■ 内置函数

- ✿ Python语言内置了若干常用的函数（在内置模块__builtins__中），在程序中可以直接使用

➤ 例如print、id、abs、max、len等

■ 标准库模块函数

- ✿ Python语言安装程序同时会安装若干标准库模块

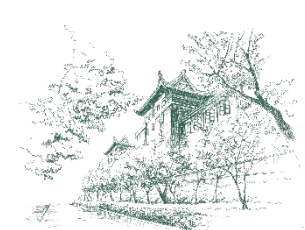
➤ 例如math、random等

- ✿ 通过import语句，可以导入标准库，然后使用其中定义的函数

➤ 例如

```
import random
```

```
one_random_number = random.randint(1,100)
```



Python函数的类型



武汉大学
WUHAN UNIVERSITY

■ 扩展库模块函数

- ✿ Python社区提供了许多其他高质量的库，如Python图像库、NumPy等
- ✿ 下载安装这些库后，通过import语句，可以导入库中的模块，然后使用其中定义的函数

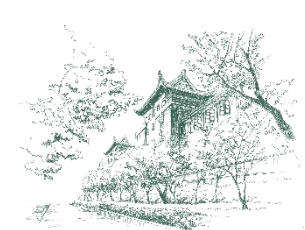
■ 用户自定义函数

- ✿ 根据解决问题的需要，程序开发人员自己设计和定义的函数
 - 例如图形绘制程序中的draw_rect函数



03

传递参数



形式参数与实际参数

■ 在图形绘制程序中定义函数draw_rect时：

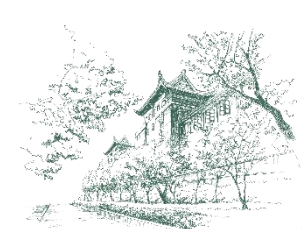
- ✿ 参数表中有height和width两个参数，实际上是两个变量，函数体内的语句需要引用两个参数的值去绘制矩形。
- ✿ 这两个参数的值是在每次调用函数draw_rect时，主程序传给其参数表中的对象或表达式的值。

■ 定义函数时所声明的参数是**形式参数**，简称**形参**。

■ 调用函数时，提供的函数所需的参数值即为**实际参数**，简称**实参**。

■ 传递参数

- ✿ 形参是变量，形式上是一个标识符；实参是一个对象或者表达式。
- ✿ 在调用函数时，通过参数传递把实参值（一个对象）的引用赋值给形参。



位置参数



- 函数调用时，默认按位置把实参顺序传递给对应的形参。
- 例如，调用函数的语句`draw_rect(3, 8)`，按位置把3和8依次赋值给形参`height`和`width`。因此，这样的参数被称为位置参数。
- ✿ 实参和形参的数量必须相同，否则会出错

```
>>> def demo(a,b,c):  
...     print(a,b,c)  
...  
>>> demo(1,2,3)  
1 2 3  
>>> demo(2,3,1)  
2 3 1  
>>> demo(1,2,3,4)  
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
TypeError: demo() takes 3 positional arguments but 4 were given
```



位置参数



■ 按位置传递的实参的数量和顺序应该和形参表一致。

✿ 考虑到绘制矩形时，可以指定构成图形的符号，增加第3个形参star。

✿ 在3次调用函数时，实参表中都增加了第3个实参。

```
# 定义函数，绘制指定大小（高height宽width）和填充符号（star）的矩形
```

```
def draw_rect(height, width, star):
```

```
    for row in range(height):
```

```
        print(star*width)
```

```
# 绘制高3宽8由*构成的矩形
```

```
draw_rect(3, 8, '*')
```

```
print()
```

```
# 绘制高3宽4由@构成的矩形
```

```
draw_rect(3, 4, '@')
```

```
print()
```

```
# 绘制高5宽3由%构成的矩形
```

```
draw_rect(5, 3, '%')
```

程序执行的结果如下：

```
*****
```

```
*****
```

```
*****
```

```
@@@@
```

```
@@@@
```

```
@@@@
```

```
%%%
```

```
%%%
```

```
%%%
```

```
%%%
```

```
%%%
```



命名参数（关键字参数）

■ 函数调用时，**可以通过名称（关键字）指定传入的参数**，这样的实参称为**关键字参数**，或称**命名参数**。

✱ 例如，绘制高3宽4由字符A构成的矩形

```
draw_rect(3, 4, star='A')
```

✱ 函数调用时，实参表中的**每一个关键字参数右边都不能有位置参数**

```
draw_rect(height=3, 4, star='A')
```

➤ 执行程序出现下面的错误：

```
draw_rect(height=3, 4, star='A')
                  ^
```

SyntaxError: positional argument follows keyword argument



命名参数（关键字参数）

■ 使用关键字参数的好处是不需要记住形参的顺序，例如下面几条语句都是正确的，并且执行后得到同样的结果。

#以下函数调用代码的功能相同

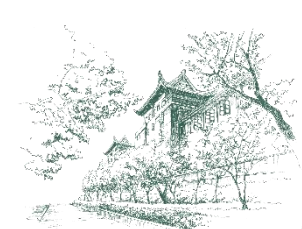
```
draw_rect(height=3, width=4, star='A')
```

```
draw_rect(3, width=4, star='A')
```

```
draw_rect(width=4, height=3, star='A')
```

```
draw_rect(width=4, star='A', height=3)
```

```
draw_rect(star='A', width=4, height=3)
```



参数的默认值（可选参数）



Python允许定义函数时在参数表中**为形参指定默认值**。

调用该函数时，如果形参没有对应的实参，则使用指定的默认值。

```
# 定义函数，绘制指定大小（高height宽width）和填充符号（star）的矩形
# 其中，star的默认值为'*'
```

```
def draw_rect(height, width, star='*'):
    for row in range(height):
        print(star*width)
```

```
# 绘制高3宽8由*构成的矩形，可以不写第3个实参
draw_rect(3, 8)
```

```
print()
```

```
# 绘制高3宽4由@构成的矩形，需要写第3个实参
draw_rect(3, 4, '@')
```

```
print()
```

```
# 绘制高5宽3由%构成的矩形，需要写第3个实参
draw_rect(5, 3, '%')
```

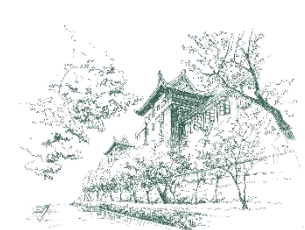
程序执行的结果如下：

```
*****
*****
*****

@@@@
@@@@
@@@@

%%%
%%%
%%%
%%%
%%%
```

必须从右往左依次为形参指定默认值，即默认值参数后面不允许出现非默认值的普通位置参数。
为形参star指定了默认值，才可以为形参width指定默认值，然后才能为形参height指定默认值，否则会提示语法错误。



参数的默认值（可选参数）

注意：

✿ 可以使用“函数名.__defaults__”随时查看函数所有默认值参数的当前值，返回值为一个元组，元素依次表示每个默认值参数的当前值。

- 多次调用函数并且不为默认值参数传递值时，默认值参数只在定义时进行一次解释和初始化，对于列表、字典这样可变类型的默认值参数，这一点可能会导致很严重的逻辑错误。
- 一般来说，要避免使用列表、字典、集合或其他可变序列作为函数参数默认值，对于左面的函数，更建议使用右面的写法（改成不可变类型的默认值参数）。

```
def demo(newitem, old_list=[]):  
    old_list.append(newitem)  
    return old_list
```

```
print(demo('x', ['a', 'b'])) # 结果为: ['a', 'b', 'x']  
print(demo('a'))           # 结果为: ['a']  
print(demo('b'))           # 结果为: ['a', 'b']  
                           # 使用上一次函数调用中修改后的列表
```

```
def demo(newitem, old_list=None):  
    if old_list is None:  
        old_list = []  
    old_list.append(newitem)  
    return old_list
```



变长参数

■ 变长参数是指在函数调用时，可以接收任意多个实参的形参。这些实参会被“打包”成一个元组或字典传递给函数，变长参数主要分为两类：

✿ *args：形参名前面有一个星号

➤ 接收额外的非关键字参数，“打包”为一个元组。

✿ **kwargs：形参名前面两个星号

➤ 接收额外的关键字参数，“打包”为一个字典，键为参数名，值为对应的参数值。

■ 变长参数必须位于形参表的末尾

✿ 这里的名称args和kwargs是习惯上使用的变长形参名称，完全可以用任意合法的标识符替换。

变长参数



使用两种变长参数的订餐程序案例

定义点餐函数

```
def order_dishes(customer_name, *dishes, number_of_people,  
                  **additional_comments):
```

此处接收3个非关键字参数

```
    print("欢迎您!", customer_name)
```

```
    print("您预定的菜单:", dishes)
```

```
    print("进餐人数是", number_of_people)
```

```
    print("您附加的说明有:", additional_comments)
```

此处接收2个关键字参数（命名参数）

模拟订餐

```
order_dishes("刘先生", "清炒菜苔", "红烧牛肉", "排骨藕汤",  
             number_of_people=2, 口味="偏清淡", 餐桌位置="僻静角落")
```

执行程序得到下面的输出:

欢迎您! 刘先生

您预定的菜单: ('清炒菜苔', '红烧牛肉', '排骨藕汤')

进餐人数是 2

您附加的说明有: {'口味': '偏清淡', '餐桌位置': '僻静角落'}

“打包”为一个元组

“打包”为一个字典



使用*和**对实参解包

■ *和**除了可以“打包”参数，还可以“解包”参数

✿ 函数调用时，*和**也可以用在实参前面，“解包”参数。

```
# 函数调用中的参数序列解包1:
# 使用一个星号 (*) 将列表或元组解包为多个参数
def func1(a,b,c):
    print(a,b,c)
    return a+b+c
args = [3,5,7]
res = func1(*args)
print("参数解包1: ",res)
```

“解包”列表为3个普通参数

```
res = func1(3,5,7)
print("使用位置参数: ",res)
```

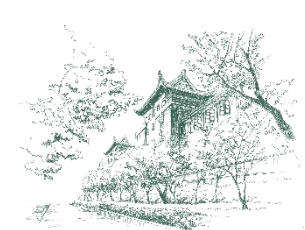
```
3 5 7
参数解包1:  15
3 5 7
使用位置参数:  15
```

```
# 函数调用中的参数解包2:
# 使用双星号 (**) 将字典键值对解包为多个关键字参数
def func2(x,y,z):
    print(x,y,z)
    return x*y*z
kwargs = {"x":2,"y":4,"z":6}
res = func2(**kwargs)
print("参数解包2: ",res)
```

“解包”字典为3个关键字参数

```
res = func2(x=2,y=4,z=6)
print("使用关键字参数: ",res)
```

```
2 4 6
参数解包2:  48
2 4 6
使用关键字参数:  48
```



强制命名参数 (Keyword-only)

■ 在带星号的参数后面申明参数会导致强制命名参数，即调用时必须显式使用命名参数传递值。

✿ 如订餐程序案例中第3个形参对应的实参：number_of_people=2

➤ 此处实参不能直接写成2，因为多个位置参数会收集为一个元组，传递给前面带星号的变长参数*dishes

✿ 若不需要带星号的变长参数，只想使用强制命名参数，可简单地使用一个星号

➤ 例如：def my_func(*, a, b, c)



04

函数返回值



函数返回值

■ 在某些情况下，函数完成数值计算或数据处理后，要把结果反馈给调用函数，即**函数返回值**。

✿ Python通过return语句返回函数值，return语句同时结束本次函数的执行。

return [表达式]

✿ 说明

- 一旦执行return语句，将直接结束函数的执行。
- 可以使用return语句返回任意类型的值给函数调用表达式。
- 如果函数没有return语句，或者有return语句但没有执行到，或者执行了不返回任何值的return语句，解释器都会认为该函数以**return None**结束，即**返回空值**。



返回简单值



✿ 举例：定义函数`circle_area`，根据半径计算圆的面积。

```
# 定义函数circle_area，根据半径计算圆的面积
def circle_area(r):
    area = 3.14*r**2
    return area
```

```
# 输入一个圆的半径，调用函数circle_area计算其面积，然后输出
radius = eval(input("输入一个圆的半径: "))
area = circle_area(radius)
print(f"该圆的面积是{area}")
```

程序一次执行的情况如下：

```
输入一个圆的半径: 5
该圆的面积是 78.5
```



返回逻辑值

✿ 举例：定义函数 `is_even` 检测任意一个大于零的整数是否为偶数，返回检测的结果，是一个布尔型的逻辑值。

```
# 定义函数is_even, 检测一个大于零的整数是否为偶数
def is_even(number):
    if number>0 and number%2==0:
        return True
    else:
        return False

# 输入一个整数, 检测其是否大于零, 且是偶数
number = int(input("输入一个整数: "))
if is_even(number):
    print("是大于零的偶数")
else:
    print("不满足条件: 大于零且为偶数")
```

运行两次程序

输入一个整数: 156
是大于零的偶数

另一次执行的情况如下:

输入一个整数: 23
不满足条件: 大于零且为偶数



包含多个return语句的函数

- 函数中可以有多条return语句，执行到哪一条return语句，哪一条return语句起作用，返回函数值并立即结束函数执行。
- 举例：定义函数判断一个数是否为素数，编写测试代码判断并输出1~99中的素数。

```
def is_prime(x):  
    if x<2: return False  
    n = 2  
    while n*n <= x:  
        if x%n == 0: return False  
        n += 1  
    return True  
  
for i in range(1, 100):  
    if is_prime(i): print(i, end=' ')
```

程序执行结果：

2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97



返回元组（包含多个值）

Python允许return语句把多个值打包到一个元组后返回。

- ✱ 如果return后面是多个逗号隔开的**数据**，则系统自动把它们“打包”到一个**元组**中，作为返回值。
- ✱ 返回值可以是任何组合类型对象，例如列表、范围对象、字典等。
- ✱ 举例：

```
def mvf():  
    pass  
    return 1, (2, 3), [4, 5, 6]
```

```
v = mvf()  
print(type(v))  
print(v)
```

```
<class 'tuple'>  
(1, (2, 3), [4, 5, 6])
```



返回元组（包含多个值）

■ 举例：定义一个函数describe，计算一组数的3个基本描述性统计量（最大值、最小值和平均值），返回这3个统计量。

```
import random

def describe(dataset):    #定义函数describe
    """计算列表或元组dataset中所有数值元素的3个统计量"""
    max_value = max(dataset)
    min_value = min(dataset)
    average = sum(dataset)/len(dataset)

    return max_value, min_value, average

# 生成一组随机数，并显示
values = [random.randint(1, 100) for _ in range(20)]
print(values)
# 调用函数describe得到这组数的统计量，并输出
max_v, min_v, avg = describe(values)
print(f"最大值: {max_v}\n最小值: {min_v}\n平均值: {avg}")
```

程序一次执行的结果如下：

```
[16, 70, 72, 19, 85, 12, 15, 16, 11, 91, 95, 97, 61, 58, 33, 90, 43, 40, 75, 27]
最大值: 97
最小值: 11
平均值: 51.3
```

函数返回一个元组（包含3个统计量）

序列解包赋值，把元组各元素分别赋值给3个变量



返回元组（包含多个值）

■ 试一试修改函数describe的定义

- ✿ 计算和返回更多统计量（上一章中学到的）。
- ✿ 然后，再定义一个函数print_describe(dataset)，能够按自己设计的样式显示一组数dataset和调用函数describe得到的若干统计量。



返回空值 (None)

■ 如果函数没有return语句，或有return语句但没有执行到，或者执行的return语句中没有表达式，解释器都会认为该函数以return None结束，即返回空值。

定义函数，不包含return语句

```
def donot_contain_return():  
    print("我有返回值吗？")
```

```
result = donot_contain_return()  
print(f"返回值是{result}，类型是{type(result)}")
```

程序执行的结果如下：

```
我有返回值吗？  
返回值是 None，类型是<class 'NoneType'>
```

定义函数，return后面没有表达式

```
def only_contain_return():  
    print("我有返回值吗？")  
    return
```

```
result = donot_contain_return()  
print(f"返回值是{result}，类型是{type(result)}")
```

程序执行的结果如下：

```
我有返回值吗？  
返回值是 None，类型是<class 'NoneType'>
```



返回列表(可变对象)



慎重返回可变对象

✿ 示例：分析下面程序的运行结果，思考函数返回值的设计时要注意的问题。

```
def f1(onelist):  
    onelist[0] = 12  
    return onelist
```

```
v1 = [1,34,56]
```

```
f1(v1)
```

```
print(v1)
```

```
pass
```

```
f1(v1)[0]=1212
```

```
pass
```

```
print(v1)
```

[12, 34, 56]

[1212, 34, 56]



05

函数定义的补充内容

■ 定义函数时，Python建议为函数增加文档字符串，从而支持自动提取函数说明功能。

✱ 文档字符串是指定义函数时紧跟函数头部增加的一个多行字符串，用来对函数的功能、参数和返回值等做解释，也被称为函数的文档注释。

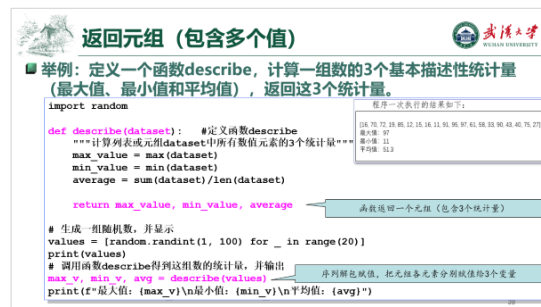
✱ 可以通过表达式 **函数名.__doc__** 来获取这个字符串

```
# 左右两侧的“__”是两个下划线字符
print("函数describe的文档字符串：", describe.__doc__)
```

程序执行后得到如下结果：

函数 describe 的文档字符串： 计算列表或元组 dataset 中所有数值元素的 3 个统计量

```
# Python的内置函数help就是通过提取函数的文档字符串来提供“帮助”
help(describe)
```



程序执行后得到如下结果：

Help on function describe in module __main__:

describe(dataset)
计算列表或元组 dataset 中所有数值元素的 3 个统计量



文档字符串



✿ 常用IDE软件在编写代码时，也可以及时提取函数的文档字符串来提供语法提示。

➤ 下图展示在Spyder中编辑代码时，编辑器自动提示已经定义的函数的头部及其文档字符串，帮助提高编写代码的效率。

The screenshot shows the Spyder IDE interface. The main editor window displays a Python script named `describe_dataset.py`. The script defines a function `describe(dataset)` that calculates the maximum, minimum, and average values of a dataset. The function's docstring is: `"""计算列表或元组dataset中所有数值元素的3个统计量"""`. The script also generates a list of random numbers and calls the `describe` function. A tooltip is visible over the `describe` function call, showing the function's signature `describe(dataset)` and its docstring. The tooltip also includes a link to click for additional help. The right sidebar shows the variable explorer and the IPython console.

```
1 import random
2
3 #定义函数describe
4 def describe(dataset):
5     """计算列表或元组dataset中所有数值元素的3个统计量"""
6     max_value = max(dataset)
7     min_value = min(dataset)
8     average = sum(dataset)/len(dataset)
9
10    return max_value, min_value, average
11
12
13 # 生成一组随机数，并显示
14 values = [random.randint(1, 100) for _ in range(20)]
15 print(values)
16 # 调用函数describe得到这组数的统计量，并输出
17 max_v, min_v, avg = describe(values)
```

describe(dataset) function
ABC describe text

describe(dataset)
计算列表或元组dataset中所有数值元素的3个统计量
Click anywhere in this tooltip for additional help

Python 3.9.13 (main, Aug 25 2022, Type "copyright", "credits" or "l
IPython 7.31.1 -- An enhanced Int
In [1]:



案例：找黑洞数

需求

✿ 编写函数计算任意位数的黑洞数。

✿ 黑洞数是指这样的整数：

➢ 由这个整数各位上的数字组成的最大数减去各位数字组成的最小数，仍然得到这个整数。

✿ 例如：

➢ 3位黑洞数是495，因为 $954 - 459 = 495$

➢ 4位数字是6174，因为 $7641 - 1467 = 6174$

➢ 9位数字是554999445，因为 $999555444 - 444555999 = 554999445$



案例：找黑洞数

实现

```
def black_holes(n):  
    '''参数n表示数字的位数，例如n=3时函数调用返回495，n=4时返回6174'''  
    #待测试数范围的起点和结束值  
    start = 10**(n-1)  
    end = 10**n  
    #依次测试每个数  
    for i in range(start, end):  
        #由整数各位上的数字组成的最大数和最小数  
        big = ''.join(sorted(str(i), reverse=True))  
        little = ''.join(reversed(big))  
        big, little = map(int, (big, little))  
        if big-little == i:  
            print(i)  
  
black_holes(3)  
black_holes(4)
```




函数注解 (Function Annotation)

■ 函数注解是指Python允许为函数的形参和返回值增加类型标注，声明形参和返回值的期望类型，以增强代码的可读性，提高程序开发的质量。

✿ 定义函数时，形参的标注形式是在形参后面加上 ": expression"

✿ 函数返回值的标注形式是在形参列表后加上 "-> expression"

```
def 函数名(参数名 : expression) -> expression :  
    函数体
```

✿ 说明

- 这些标注可以是任何有效的 Python 表达式。
- 标注的存在不会改变函数的语义。



函数注解 (Function Annotation)



武汉大学
WUHAN UNIVERSITY

■ 举例：定义函数count，统计指定数在一组数中出现的次数。

```
import random
```

```
# 定义函数count
```

```
def count(dataset:list, k:int)->int:  
    """统计k在dataset中出现的次数"""  
    return dataset.count(k)
```

函数注解（类型标注）提醒调用函数时所期望的实参类型以及函数返回值类型

```
# 生成一组随机数，并打印出来
```

```
numbers = [random.randint(10, 20) for _ in range(20)]  
print(numbers)
```

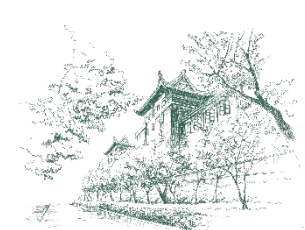
```
# 输入一个整数。然后调用count函数统计该数出现的次数
```

```
number = int(input("输入要找的数: "))
```

```
print(f"{number}在数据集中出现了{count(numbers, number)}次。")
```

程序一次执行的情况如下：

```
[11, 11, 17, 19, 20, 14, 14, 17, 11, 19, 13, 11, 16, 19, 18, 14, 20, 20, 14, 19]  
输入要找的数: 11  
11 在数据集中出现了 4 次。
```



函数注解 (Function Annotation)



武汉大学
WUHAN UNIVERSITY

■ 函数注解声明的只是“期望”，Python解释器并不严格要求实参的类型必须符合这个期望。

```
# 定义函数fuzzy_expectation
def fuzzy_expectation(one:int, another:int) ->int:
    """计算整数one和another的和"""
    return one + another
```

```
# 实际上，调用fuzzy_expectation可以传递各种类型的值
try_1 = fuzzy_expectation(123, 456)
print(type(try_1), try_1)
try_2 = fuzzy_expectation(1.23, 45.6)
print(type(try_2), try_2)
try_3 = fuzzy_expectation("fuzzy", " expectation")
print(type(try_3), try_3)
```

程序中3次函数调用传递的参数类型分别是int、float和str，都支持“+”运算，得到的返回值类型也分别是int、float和str

程序执行的结果如下：

```
<class 'int'> 579
<class 'float'> 46.83
<class 'str'> fuzzy expectation
```

想一想，函数调用fuzzy_expectation("whu", 666)是否正确？如果出错，错在哪里？如何改正此错误？



武汉大学
WUHAN UNIVERSITY

06

传递列表



传递列表（可变对象）

- 列表与内置基本数据类型、元组和字符串的区别：列表对象是可变对象。
 - ✿ 在调用函数时传递列表就有非常不一样的特点：列表的元素在函数体内可以增减和修改。了解这一特点可以避免编写的程序出现“奇怪的”错误。
 - ✿ 当然，可以利用列表做函数参数的特点，多次调用函数访问和修改同一个对象。例如实现来访者登记簿程序（参见后面的程序设计案例）。
 - ✿ 举例：通过定义两个函数，分别用不可变（int）对象做参数和可变对象（list列表）做参数，进行增量修改，对比看一下两个函数的使用效果和区别。



传递列表（可变对象）

■ 举例：定义两个函数，分别对一个量和一组量（列表）的值进行增量修改。

```
# 定义函数increment_count
def increment_count(count:int, increment:int):
    """使count的值增加increment"""
    print(f"执行增量前, count的值是{count}, 标识是{id(count)}")
    count += increment
    print(f"执行增量后, count的值是{count}, 标识是{id(count)}")

# 定义函数increment_counts
def increment_counts(counts:list, increment:int):
    """使counts中所有元素的值增加increment"""
    print(f"执行增量前, counts的值是{counts}, 标识是{id(counts)}")
    for i in range(len(counts)):
        counts[i] += increment
    print(f"执行增量后, counts的值是{counts}, 标识是{id(counts)}")
```



传递列表 (可变对象)

对比实验

```
count_value = 100
```

```
print(f"调用增量函数前, count_value的值是{count_value}, 标识是{id(count_value)}")
```

```
increment_count(count_value, 10) # 把整数100的引用传递给形参count
```

```
print(f"调用增量函数后, count_value的值是{count_value}, 标识是{id(count_value)}")
```

```
print()
```

```
count_values = [100, 200, 300]
```

```
print(f"调用增量函数前, count_values的值是{count_values}, 标识是{id(count_values)}")
```

```
increment_counts(count_values, 10) # 把列表对象[100, 200, 300]的引用传递给形参counts
```

```
print(f"调用增量函数后, count_values的值是{count_values}, 标识是{id(count_values)}")
```

程序执行的结果如下:

调用增量函数前, count_value 的值是 100, 标识是 2198882964944
执行增量前, count 的值是 100, 标识是 2198882964944
执行增量后, count 的值是 110, 标识是 2198882965264
调用增量函数后, count_value 的值是 100, 标识是 2198882964944

对比1: 整数100是不可变对象, 函数操作对100没有影响, 也不会改变count_value对100的引用。执行增量后, count引用了一个新对象, 其值为110, 其标识不同于100的标识

调用增量函数前, count_values 的值是[100, 200, 300], 标识是 2198967138304
执行增量前, counts 的值是[100, 200, 300], 标识是 2198967138304
执行增量后, counts 的值是[110, 210, 310], 标识是 2198967138304
调用增量函数后, count_values 的值是[110, 210, 310], 标识是 2198967138304

对比2: 列表[100, 200, 300]是可变对象, 函数操作对其有影响 (修改了元素的值)。执行增量前后, count_values和counts引用的都是这个列表对象



示例：随机混排给定列表的元素值

■ 用可变对象作为参数，随机混排给定列表的元素值。

```
import random
# 随机混排列表lst中的元素
def shuffle(lst):
    n = len(lst)
    for i in range(n):
        j = random.randint(0, n-1)  #生成与lst[i]交换值的元素的下标
        lst[i], lst[j] = lst[j], lst[i]
# 对shuffle函数进行测试
def test_shuffle():
    v = [1,2,3,4,5]
    shuffle(v)  #调用shuffle函数后，实参v引用的列表被修改（洗牌）了
    print(v)

test_shuffle()
```



案例：来访者登记簿程序

■ 利用列表做函数参数的特点，实现模拟来访者登记簿生成过程。

✿ 定义函数 `check_in` 登记来访者的姓名和电话，并把记录保存到一个来访者列表。

定义登记来访者信息的函数

```
def check_in(visitors:list):
```

```
    """记录来访者输入的姓名和联系电话，追加到visitors"""
```

```
    name = input("请输入姓名: ")
```

```
    tel = input("联系电话: ")
```

```
    visitors.append((name, tel)) # 来访者的姓名和联系电话构成一个元组，并追加到列表
```

```
visitor_book = [ ] # 定义列表用来保存即将到来的来访者信息
```

假设有3位来访者的信息需要登记，`_`用作临时循环变量

```
for _ in range(3):
```

```
    check_in(visitor_book)
```

```
print(visitor_book)
```

利用列表做函数参数的特点，多次调用函数访问和修改同一个列表对象（来访者登记簿）

变量`visitor_book`引用的列表用来保存来访者登记信息，程序开始执行时是空的，每次调用函数`check_in`，把该列表传递给形参`visitors`，在函数内接收来访者信息后添加到列表中，经过3次调用完成了对3位来访者信息的登记。

程序一次执行的情况如下：

请输入姓名：王涛

联系电话：12341234

请输入姓名：苏越

联系电话：56785678

请输入姓名：屈鑫

联系电话：90129012

[('王涛', '12341234'), ('苏越', '56785678'), ('屈鑫', '90129012')]



武汉大学
WUHAN UNIVERSITY

07

实践指导



应用案例：交互式图形绘制程序

■ 实现交互式图形绘制程序

✿ 基于标准库turtle模块的绘图功能编写一个简单的交互式图形绘制程序，可以让用户选择需要绘制的图形的类型、大小和边框宽度。

✿ 大致设计思路：

- (1) 为绘制每一种类型的图形定义对象的绘制函数；
- (2) 定义一个函数显示功能菜单，并提示用户选择要绘制的图形类型；
- (3) 在主程序中初始化绘图窗口，构造一个循环，每次调用菜单函数进行询问，根据用户的绘图需求调用相应的图形绘制函数，直至用户不再绘图，选择退出程序。

```
# paint_shape.py

"""
交互式图形绘制程序
"""

# 导入turtle模块
import turtle
```



应用案例：交互式图形绘制程序



武汉大学
WUHAN UNIVERSITY

■ 实现交互式图形绘制程序

```
# 定义功能菜单函数
def menu():
    """显示功能菜单，提示用户选择要绘制的图形，返回用户的选择"""
    print("=====")
    print("我会绘制下面几种图形：")
    print("[1]矩形")
    print("[2]圆")

    tip = "请输入1或2告诉我要绘制的图形（输入0退出程序）："
    shape_type = -1

    # 提示用户输入有效的图形类型数字编号或0，并做有效性检查
    while shape_type not in (0, 1, 2):
        shape_type = int(input(tip))

    return shape_type
```



应用案例：交互式图形绘制程序



武汉大学
WUHAN UNIVERSITY

■ 实现交互式图形绘制程序

```
# 定义绘制矩形的函数
def draw_rect(height, width, size):
    tt.reset()
    tt.pensize(size)

    tt.forward(width // 2)
    tt.left(90)
    tt.forward(height)
    tt.right(90)
    tt.backward(width)
    tt.right(90)
    tt.forward(height)
    tt.left(90)
    tt.forward(width // 2)
```



应用案例：交互式图形绘制程序



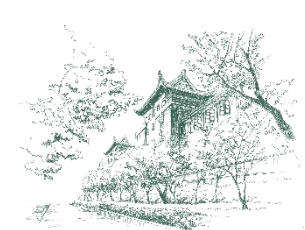
武汉大学
WUHAN UNIVERSITY

■ 实现交互式图形绘制程序

```
# 定义绘制矩形的准备函数
def to_draw_rect():
    height = int(input("输入矩形高度: "))
    width = int(input("宽度: "))
    size = int(input("边框宽度 (1~10): "))

    draw_rect(height, width, size)
```

to_draw_rect() 函数主要是通过询问方式让用户确定矩形大小和样式的参数值，然后调用draw_rect() 绘制矩形函数



应用案例：交互式图形绘制程序



武汉大学
WUHAN UNIVERSITY

■ 实现交互式图形绘制程序

定义绘制圆形的函数

```
def draw_circle(radius, size):  
    tt.reset()  
    tt.pensize(size)  
  
    tt.circle(radius)
```

定义绘制圆形的准备函数

```
def to_draw_circle():  
    radius = int(input("输入圆的半径: "))  
    size = int(input("边框宽度 (1~10): "))  
  
    draw_circle(radius, size)
```

to_draw_circle() 函数主要是通过询问方式让用户确定圆的半径和样式的参数值，然后调用draw_circle() 绘制圆形函数



应用案例：交互式图形绘制程序



武汉大学
WUHAN UNIVERSITY

■ 实现交互式图形绘制程序

```
if __name__ == '__main__':  
    tt = turtle.Turtle()  
    tt.hideturtle()  
  
    print("欢迎使用图形绘制程序！")  
    while True:  
        shape_type = menu()  
  
        if shape_type == 1:  
            to_draw_rect()  
        elif shape_type == 2:  
            to_draw_circle()  
        else:  
            break  
  
    tt.screen.bye()  
    print("Bye")
```



应用案例：交互式图形绘制程序

■ 实现交互式图形绘制程序

✦ 试一试，对这个程序的功能从多方面进行扩展：

- (1) 能够绘制更多的图形类型，例如三角形、平行四边形或菱形等；
- (2) 支持更多参数或样式，例如图形的坐标（位置）、边框颜色、是否填充和选择填充色等；
- (3) 画布设置，例如背景色、在每次绘图前是否清空画布和是否显示海龟等。



小结



- 程序模块化采用“分而治之，自顶向下”的方法解决问题，函数是实现Python程序模块化的基础方法，一个.py程序文件称为一个模块。
- 用def语句定义函数，函数体需采用缩进书写规则。
- 函数定义并不会自动执行函数体，只有当函数被调用时才会执行。
- 函数调用时，通过参数传递把实参值（一个对象）的引用赋值给形参。
 - ✳ 位置参数、关键字参数（命名参数）、默认值参数（可选参数）和变长参数。
- 可以用return语句立即结束函数的执行，并返回数据。
 - ✳ 函数返回值可以是简单值、逻辑值、元组（多值）和空值（None）。
- 函数定义时，可以添加文档字符串和函数注解。
- 要注意列表做函数参数时，列表对象的可变性对函数的影响。
- 可以使用标准库turtle模块的绘图功能实现交互式图形绘制程序。



武汉大学

WUHAN UNIVERSITY

谢谢

THANK YOU

